

AI Coding for Researchers — Cheat Sheet

Based on my own personal experience with LLM agentic coding.

Ken Krige, ITS Utrecht University

This is Claude-specific. They all have equivalents — just ask any agent what its equivalent is.

REMEMBER

1. Attention reads **every token at once**. Cost grows as n^2 , so the window has to be capped.
2. The model remembers **nothing**. The whole conversation gets re-sent for every turn, re-processed for every token.
3. Keep what matters in **files, issues and PRs**. Not in the chat.

CONTEXT LEVERS

- `/context` shows what's filling the window. Run it often. It's an eye-opener.
- `/clear` wipes it. Start fresh for an unrelated task.
- `/compact` summarises the older turns. Do it at a milestone, not mid-investigation. It loses things.
- `.gitignore` keeps the junk out: data dumps, generated files, **notebooks with output cells**, vendored code. Claude respects it by default.
- Keep `CLAUDE.md` lean. Every line is paid for on every turn of every session.
- **Subagents** and `/explore` work in a separate window. Only the answer comes back, not the 50 file dumps.

WORKING WITH THE AGENT

- Agents are fantastic at **routine tasks**, even difficult routine tasks. Using github versioning properly got much easier. Just tell the agent what you want.
- Agents are fantastic at **translating**, language to code, code to language. Agents are not good at interpreting pictures. Claude is terrible, GPT is better. None are good.
- Explain the desired **output** in words. Give it an example if you can. Ask the agent to explain its algorithm back to you.
- If you are doing **original research**, do not expect an LLM to do it for you. Its knowledge and training are based on existing literature. They can be stubborn about this. Force it to stick to your idea, let it implement the detail. If it goes too easily, question how original your research is.
- **Agree the plan** before any code.
- When it goes badly, talk it through in **pseudo-code**. A good halfway stop before touching the code.
- Ask for an **HTML diagram** when the explanations get long. Especially when it starts drawing pictures in ASCII.
- Give it a way to **check its own work**: a test, or a way to run the app.
- Be **specific**. Point at file paths. Give it an example.
- Learn its lingo and use it.

SCOPE & SAFETY NET

- **One issue, one branch, one session**. Keeps the context bounded and the diff small enough to read.
- Small scope means the whole task **fits in the window**.
- **CI is the safety net**. Two PRs can each pass on their own and still break main. Each agent only saw its own slice.
- When CI gets expensive, **scope it down** (changed files only, fast, slow, weekly tiers). Don't switch it off.
- Decide who approves **the merge**.
- Always do **code reviews**. Use different models and people.
- Have a **testing environment** separate from your production/research data. Agents are good at generating artificial data. Always run the code on **test data** first.

KNOWLEDGE THAT SURVIVES `/clear`

- Agents read `CLAUDE.md` and the READMEs at the **start of every session**. Treat the docs like code.
- Read **agent generated design docs** very carefully. Agents use them as their bible.
- `DECISIONS.md`: every real choice, **and the options you rejected**. Stops you re-arguing it next month.
- Write a **design doc before the code**. It captures what you chose not to build. Use openspec or similar.
- Post the **conversation transcript** into an issue comment. Sessions get wiped, issues don't.
- **Markdown for everything**: specs, decisions, docs, even these slides. Diffable, and the agent can read it.

ROLES, PERSONAS & MODEL CHOICE

- **Roles** (the job): junior dev, architect, reviewer, cleaner.
- **Personas** (how it thinks): meticulous, creative, maverick. A meticulous reviewer catches the off-by-one. A maverick asks if you should be building this at all.

Large

when a mistake is silent

Medium

plumbing, ~60% of it

Small

cleanups